ENTERPRISE SECURITY ASSESSMENT LAB

EXCESSIVE DATA EXPOSURE AND HIDDEN FIELD REVIEW

API response inspection for unnecessary fields, hidden attributes, and sensitive metadata

Field	Details
Module	Sub-Project 2 - API Security Testing Lab
Document Type	Individual API Security Methodology PDF
Phase / Test Case	Phase 5 - Data Exposure Testing
Prepared For	GitHub Portfolio Documentation
Prepared By	Jagriti Banerjee
Repository	Enterprise-Security-Assessment-Lab
GitHub Filename	05-excessive-data-exposure-testing-methodology-report.pdf

This document is a professional, evidence-driven explanation of one API security methodology section. It is designed for portfolio review and contains only authorized lab-based testing context. Sensitive token values, account data, cookies, and private identifiers are redacted or intentionally represented with placeholders.

1. Section Overview

Area	Details
Phase	Phase 5 - Data Exposure Testing
Objective	Review API responses for fields that are not required by the frontend but are still returned to the client.
Primary Result	Evidence captured and methodology documented
GitHub Folder	02-api-security/reports/methodology/

Why this section matters

API data exposure is often missed because the UI hides fields while the raw API still returns them. A professional API review must inspect the response body, not only the visible web page.

2. Evidence Embedded in This PDF

Evidence 1: 07-excessive-data-exposure-hidden-fields.png

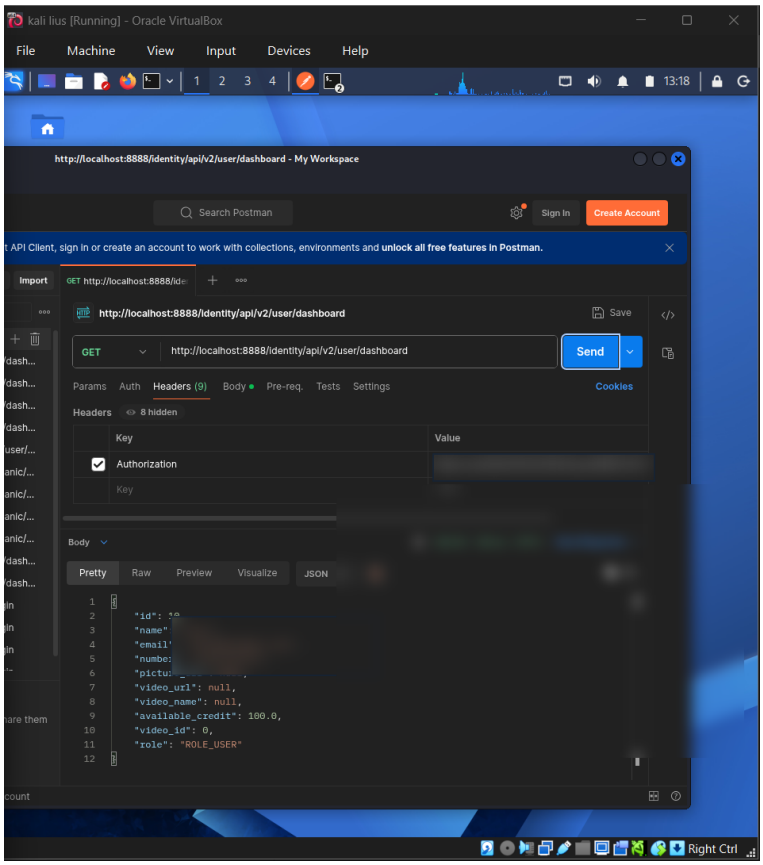

Figure 1 - Evidence showing API response review for hidden or excessive fields.

Evidence showing API response review for hidden or excessive fields.

3. Command and Workflow Used

```
GET /api/user/profile
Authorization: Bearer <REDACTED_TOKEN>

# Fields reviewed in API responses
internal IDs
roles
email fields
hidden attributes
debug fields
sensitive metadata

# Example using curl and jq
curl -s -H "Authorization: Bearer <REDACTED_TOKEN>" \
  http://127.0.0.1:8888/api/user/profile | jq
```

Step-by-step workflow

- Sent authenticated API requests and reviewed raw JSON responses.
- Compared returned fields against what a frontend would realistically need.
- Looked for internal IDs, roles, sensitive metadata, debug keys, or hidden attributes.
- Captured evidence of the response structure and field exposure review.

4. Professional Interpretation

API data exposure is often missed because the UI hides fields while the raw API still returns them. A professional API review must inspect the response body, not only the visible web page.

5. Security Risk and Impact

Excessive data exposure can leak internal model structure, user metadata, privilege indicators, emails, system IDs, or business-sensitive fields. Attackers can combine this with BOLA, enumeration, or privilege escalation attempts.

6. Recommended Remediation and Hardening

- Return only fields required by the client.
- Use DTOs/serializers instead of returning raw database models.
- Apply field-level authorization for sensitive attributes.
- Review API responses manually during security testing.
- Avoid exposing debug fields, internal flags, roles, and hidden metadata to normal users.

7. Framework Mapping

Framework Area	Mapping
OWASP API	API3: Broken Object Property Level Authorization
Security Control	Response minimization and field-level authorization

Framework Area	Mapping
Evidence Result	Data exposure review completed

8. Sanitized Output Style for GitHub

When documenting this evidence publicly, use placeholders instead of live values. The goal is to prove methodology and security understanding without exposing secrets or private data.

```
Authorization: Bearer <REDACTED_TOKEN>
Cookie: <REDACTED_COOKIE>
email: lab-user@example.com
password: REDACTED
object_id: <CONTROLLED_LAB_ID>
redirect_uri: <REDACTED_OR_LAB_CALLBACK>
```

9. GitHub Redaction Checklist

- Bearer tokens and JWT values must be blurred or replaced with <REDACTED_TOKEN>.
- Email addresses, phone numbers, user IDs, and object identifiers should be blurred when they are not required for understanding the evidence.
- Authorization headers, cookies, session values, OAuth codes, refresh tokens, access tokens, and client secrets must never be visible in public GitHub screenshots.
- If a screenshot contains personally identifiable data, crop the view or blur the affected rows before publication.

10. Publication Note

This report is designed for GitHub portfolio use. It describes authorized lab testing only and avoids production claims. The embedded screenshots have been treated as lab evidence and should remain redacted before upload.